



**ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΕΙΡΑΙΩΣ**

**ΣΧΟΛΗ ΤΕΧΝΟΛΟΓΙΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ  
ΤΗΛΕΠΙΚΟΙΝΩΝΙΩΝ**

**ΤΜΗΜΑ ΠΛΗΡΟΦΟΡΙΚΗΣ**

**3η Εργασία ΣΥΓΧΡΟΝΑ ΘΕΜΑΤΑ ΤΕΧΝΟΛΟΓΙΑΣ ΛΟΓΙΣΜΙΚΟΥ -  
ΛΟΓΙΣΜΙΚΟ ΓΙΑ ΚΙΝΗΤΕΣ ΣΥΣΚΕΥΕΣ**

**ASP.NET Core MVC Architecture**

**ΟΜΑΔΑ:**

**ΑΝΑΣΤΑΣΙΑ ΜΑΝΩΛΑ Π21091**

**ΔΗΜΗΤΡΙΟΣ ΦΩΤΙΑΔΗΣ Π21183**

**ΜΑΤΙΝΑ ΠΑΠΑΔΑΚΟΥ Π21127**

# Τεχνικό Εγχειρίδιο

## 1. HomeController

Ο HomeController διαχειρίζεται την κύρια σελίδα της εφαρμογής και την προβολή των πληροφοριών χρήστη, όταν είναι συνδεδεμένος.

### Index():

```
// GET: Login
public IActionResult Index()
{
    return View();
}
```

- Επιστρέφει τη σελίδα σύνδεσης (Login), μέσω του αντίστοιχου View.
- Εμφανίζει τη φόρμα όπου ο χρήστης μπορεί να εισάγει username και userId.

### Login(string username, int userId):

```
// Καθαρισμός της συνεδρίας πριν τη σύνδεση
HttpContext.Session.Clear();
```

Η Login:

- Ελέγχει αν τα στοιχεία εισόδου (username και userId) είναι έγκυρα.
- Αναζητά τον χρήστη στη βάση δεδομένων:

```
// Αναζητά τον χρήστη στη βάση δεδομένων
var user = await _context.Users.FirstOrDefaultAsync(u => u.Username == username && u.UserId == userId);
```

Αν βρεθεί ο χρήστης:

- Ελέγχει σε ποια κατηγορία ανήκει ο χρήστης (π.χ. Πελάτης, Πωλητής, Διαχειριστής).
- Αποθηκεύει τις πληροφορίες για τον χρήστη στη συνεδρία (HttpContext.Session).
- Ανακατευθύνει στις κατάλληλες λεπτομέρειες του χρήστη

```
// Αναζητά τον χρήστη στη βάση δεδομένων
var user = await _context.Users.FirstOrDefaultAsync(u => u.Username == username && u.UserId == userId);

if (user != null)
{
    // Ο χρήστης βρέθηκε, εμφάνιση των πληροφοριών του
    if (_context.Clients.Any(c => c.UserId == user.UserId))
    {
        // Αν είναι πελάτης
        var client = await _context.Clients.Include(c => c.User).FirstOrDefaultAsync(c => c.UserId == user.UserId);
        // Αποθήκευση του τύπου χρήστη στη συνεδρία
        HttpContext.Session.SetString("UserType", "Clients");
        HttpContext.Session.SetString("UserId", user.UserId.ToString()); // Αποθηκεύουμε το UserId στη συνεδρία
        // Καταγραφή για την αποθήκευση του τύπου χρήστη στη συνεδρία
        Console.WriteLine("UserType set to 'Clients'");
        return RedirectToAction("Details", "Clients", new { id = client.UserId });
    }
    else if (_context.Sellers.Any(s => s.UserId == user.UserId))
    {
        // Αν είναι πωλητής
        var seller = await _context.Sellers.Include(s => s.User).FirstOrDefaultAsync(s => s.UserId == user.UserId);
        HttpContext.Session.SetString("UserType", "Sellers");
        HttpContext.Session.SetString("UserId", user.UserId.ToString());
        Console.WriteLine("UserType set to 'Seller'");
        return RedirectToAction("Details", "Sellers", new { id = seller.UserId });
    }
    else if (_context.Admins.Any(a => a.UserId == user.UserId))
    {
        // Αν είναι admin
        var admin = await _context.Admins.Include(a => a.User).FirstOrDefaultAsync(a => a.UserId == user.UserId);
        HttpContext.Session.SetString("UserType", "Admin");
        HttpContext.Session.SetString("UserId", user.UserId.ToString());
        Console.WriteLine("UserType set to 'Admin'");
        return RedirectToAction("Details", "Admin", new { id = admin.UserId });
    }
    else
    {
        // Άλλες κατηγορίες χρηστών
        TempData["Message"] = "Ο χρήστης δεν ανήκει σε καμία κατηγορία.";
        return RedirectToAction("Index");
    }
}
```

Αν δεν βρεθεί ο χρήστης:

- Εμφανίζεται μήνυμα λάθους στην φόρμα σύνδεσης:

```
}  
else  
{  
    // Ο χρήστης δεν βρέθηκε  
    ModelState.AddModelError("", "Λάθος όνομα χρήστη ή κωδικός.");  
    return View("Index");  
}
```

## Logout():

```
// Αποσύνδεση (Logout) χρήστη  
public IActionResult Logout()  
{  
    // Διαγραφή όλων των δεδομένων της συνεδρίας  
    HttpContext.Session.Clear();  
    return RedirectToAction("Index", "Login"); // Επιστροφή στην σελίδα σύνδεσης  
}
```

- Καθαρίζει τη συνεδρία και αποσυνδέει τον χρήστη.
- Ανακατευθύνει στη σελίδα σύνδεσης (Index).

## Παράδειγμα:

Κατά τη σύνδεση του χρήστη, ο Controller λαμβάνει τα δεδομένα εισόδου από τη φόρμα σύνδεσης και χρησιμοποιεί το μοντέλο (Admin, Client, Seller, User) για να αναζητήσει τα στοιχεία του χρήστη στη βάση δεδομένων. Αν η αναζήτηση είναι επιτυχής και τα στοιχεία είναι έγκυρα, ο χρήστης ανακατευθύνεται στο View της κεντρικής σελίδας. Αν η αναζήτηση αποτύχει (π.χ. μη έγκυρα στοιχεία ή μη ύπαρξη του χρήστη), εμφανίζεται μήνυμα λάθους στη φόρμα σύνδεσης μέσω του View.

## Σύνδεση

Όνομα Χρήστη

jsmith

Κωδικός (UserId)

.

Σύνδεση

MyApp

Πελάτες ▾

Διαχειριστές ▾

Πωλητές ▾

## Καλωσήρθατε!

Είστε συνδεδεμένος ως Πελάτης.

## 2. ClientsController

Ο ClientsController διαχειρίζεται όλες τις λειτουργίες που αφορούν τους πελάτες, όπως την προβολή, δημιουργία, επεξεργασία και διαγραφή στοιχείων πελατών. Εξασφαλίζει ότι τα δεδομένα των πελατών αποθηκεύονται και ενημερώνονται σωστά στη βάση δεδομένων, ενώ

εφαρμόζει ελέγχους πρόσβασης για να διασφαλίσει ότι μόνο οι εξουσιοδοτημένοι χρήστες μπορούν να εκτελέσουν συγκεκριμένες λειτουργίες.

### IsClient() και IsSeller():

```
private bool IsClient()
{
    var userType = HttpContext.Session.GetString("UserType");
    return userType == "Clients";
}

private bool IsSeller()
{
    var userType = HttpContext.Session.GetString("UserType");
    return userType == "Sellers";
}
```

Και οι δύο μέθοδοι είναι βοηθητικοί και χρησιμοποιούνται για να ελέγξουν τον τύπο χρήστη που είναι συνδεδεμένος στην εφαρμογή.

Η μέθοδος IsClient() ελέγχει αν ο συνδεδεμένος χρήστης είναι πελάτης (Client):

Λειτουργία:

- Χρησιμοποιεί τη Session του χρήστη για να διαβάσει μια τιμή που ονομάζεται UserType.
- Συγκρίνει την τιμή αυτή με τη συμβολοσειρά "Clients".
- Αν το UserType είναι "Clients", επιστρέφει true, που σημαίνει ότι ο χρήστης είναι πελάτης.

Το αντίστοιχο συμβαίνει και στην μέθοδο IsSeller().

### Index():

Πραγματοποιείται το ερώτημα 1.1 (Πελάτης) Προβολή Λογαριασμού Πελάτη

```
// GET: Clients
public async Task<IActionResult> Index()
{
    if (!IsClient())
    {
        TempData["AlertMessage"] = "You need to log in as Client!";
        return RedirectToAction("LogIn", "Home");
    }
}
```

- Εδώ χρησιμοποιείται η μέθοδος IsClient() για να ελεγχθεί αν ο χρήστης που προσπαθεί να προσπελάσει τη σελίδα **Clients** είναι συνδεδεμένος ως πελάτης.

- Αν δεν είναι πελάτης, επιστρέφεται μήνυμα σφάλματος και ο χρήστης ανακατευθύνεται στη σελίδα σύνδεσης (**Login**).

#### MVC Λογική:

- **Controller:** Ελέγχει την πρόσβαση στον χρήστη.
- **View:** Χρησιμοποιείται το TempData["AlertMessage"] για την εμφάνιση μηνύματος στο UI.

```
else
{
    var userId = HttpContext.Session.GetString("UserId");

    if (string.IsNullOrEmpty(userId))
    {
        ViewData["AlertMessage"] = "You need to log in!";
        return RedirectToAction("Index", "Home");
    }
}
```

- Η εφαρμογή διαβάζει από το **Session** την τιμή του UserId, που έχει αποθηκευτεί κατά τη σύνδεση του χρήστη.
- Αν το UserId είναι κενό, ο χρήστης ανακατευθύνεται στην αρχική σελίδα, καθώς δεν είναι συνδεδεμένος.

#### MVC Λογική:

- **Controller:** Ελέγχει αν υπάρχει έγκυρη συνεδρία χρήστη.
- **View:** Αν η συνεδρία δεν είναι έγκυρη, εμφανίζεται μήνυμα για να ειδοποιηθεί ο χρήστης.

```
var client = await _context.Clients
    .Include(c => c.User)
    .Where(c => c.UserId.ToString() == userId)
    .FirstOrDefaultAsync();

if (client == null)
{
    return NotFound();
}

return View(client);
}
```

#### Ανάκτηση δεδομένων από την Βάση:

- **\_context.Clients:** Πρόκειται για το αντικείμενο **DbSet<Client>**, που εκπροσωπεί τον πίνακα **Clients** στη Βάση Δεδομένων.
- **Include(c => c.User):** Ενώνει (joins) τον πίνακα **Clients** με τον πίνακα **Users** για να φέρει και τα δεδομένα του σχετιζόμενου χρήστη (User).

- **Where(c => c.UserId.ToString() == userId):** Εφαρμόζεται φίλτρο για να φέρει μόνο τον πελάτη που αντιστοιχεί στο UserId της συνεδρίας.
- **FirstOrDefaultAsync():** Επιστρέφει την πρώτη εγγραφή που ταιριάζει με το φίλτρο (ή null αν δεν υπάρχει).

#### MVC Λογική:

- **Model:** Η αναζήτηση δεδομένων γίνεται μέσω του Entity Framework, που χειρίζεται τις εγγραφές της Βάσης Δεδομένων.
- **Controller:** Ο Controller καθορίζει ποια δεδομένα θα ανακτηθούν (μόνο του συνδεδεμένου πελάτη).

```

        if (client == null)
        {
            return NotFound();
        }

        return View(client);
    }

```

- Αν δεν βρεθεί πελάτης με το συγκεκριμένο **UserId**, επιστρέφεται HTTP Status Code **404 (Not Found)**.

Για το if:

#### MVC Λογική:

- Ο **Controller** χειρίζεται περιπτώσεις όπου τα δεδομένα δεν υπάρχουν, επιστρέφοντας κατάλληλη απόκριση στον χρήστη.
- Αν βρεθεί πελάτης, τα δεδομένα του περνιούνται στο **View**, το οποίο θα τα εμφανίσει στον χρήστη.
- Τα δεδομένα αυτά περιλαμβάνουν πληροφορίες από τον πίνακα **Clients** καθώς και τις σχετιζόμενες πληροφορίες από τον πίνακα **Users**.

Για το return:

#### MVC Λογική:

- **Model:** Τα δεδομένα έρχονται από το Entity Framework.
- **View:** Το View δέχεται τα δεδομένα και τα εμφανίζει στον χρήστη.
- **Controller:** Παίζει το ρόλο της γέφυρας ανάμεσα στο Model και το View.

### Παράδειγμα:

Αφού ο χρήστης συνδεθεί με τα διαπιστευτήρια username: jsmith και password: 2, τα στοιχεία του θα ανακτηθούν από τη βάση δεδομένων μέσω των Models και θα υποβληθούν σε επεξεργασία από τον Controller. Στη συνέχεια, με τη βοήθεια των Views, θα εμφανιστεί η κατάλληλη σελίδα στον χρήστη.

## Στοιχεία Πελάτη

| Username | First Name | Last Name | AFM       | Phone Number |
|----------|------------|-----------|-----------|--------------|
| jsmith   | Jane       | Smith     | 123456789 | 6971234567   |

### Create():

Πραγματοποιείται το ερώτημα 3.1 (Πωλητής) Καταχώρηση Νέου Πελάτη

```
// GET: Clients/Create
public IActionResult Create()
{
    if (!IsSeller())
    {
        TempData["AlertMessage"] = "You need to log in as Seller!";
        return RedirectToAction("LogIn", "Home");
    }
    ViewData["UserId"] = new SelectList(_context.Users, "UserId", "UserId");
    return View();
}
```

### Έλεγχος Πρόσβασης:

- Χρησιμοποιείται η μέθοδος IsSeller για να ελεγχθεί αν ο χρήστης που επιχειρεί να δημιουργήσει νέο πελάτη είναι συνδεδεμένος ως Seller.
- Αν ο χρήστης δεν είναι Seller, αποθηκεύεται ένα μήνυμα σφάλματος στο TempData["AlertMessage"] και ο χρήστης ανακατευθύνεται στη σελίδα σύνδεσης (LogIn).

### MVC Λογική:

- Ο Controller χειρίζεται τον έλεγχο πρόσβασης, προστατεύοντας τη λειτουργία δημιουργίας μόνο για συγκεκριμένο τύπο χρηστών (π.χ., πωλητές).

### Δημιουργία Δεδομένων για τη Φόρμα

- \_context.Users: Πρόκειται για το DbSet των χρηστών (Users) που αντιστοιχεί στον πίνακα Users στη Βάση Δεδομένων.

- new SelectList(...): Δημιουργεί μια λίστα επιλογών (dropdown list) για να εμφανιστεί στη φόρμα.
  - Πηγή δεδομένων είναι οι εγγραφές του πίνακα Users.
  - UserId χρησιμοποιείται ως τιμή (value) και ως ετικέτα (label) για κάθε επιλογή.

### MVC Λογική:

- Model: Τα δεδομένα για το dropdown list αντλούνται από τη Βάση Δεδομένων.
- Controller: Προετοιμάζει τα δεδομένα και τα περνάει στο View μέσω του ViewData.

### Create ([Bind("Afm,PhoneNumber,User")] Client client):

Πραγματοποιείται το ερώτημα 3.1 (Πωλητής) Καταχώρηση Νέου Πελάτη

Η μέθοδος **POST: Create** αποθηκεύει έναν νέο πελάτη (**Client**) και τον συνδέει με έναν νέο χρήστη (**User**) στη βάση δεδομένων.

```
// POST: Clients/Create
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("Afm,PhoneNumber,User")] Client client)
{
    if (ModelState.IsValid)
    {
        var newUser = new User
        {
            FirstName = client.User.FirstName,
            LastName = client.User.LastName,
            Username = client.User.Username,
        };

        client.UserId = newUser.UserId;

        _context.Clients.Add(client);
        await _context.SaveChangesAsync();

        TempData["SuccessMessage"] = $"Ο χρήστης καταχωρήθηκε με επιτυχία με ID: {client.UserId}";
    }
    else
    {
        TempData["SuccessMessage"] = $"Ο χρήστης δεν καταχωρήθηκε!";
    }

    return View(client);
}
```

### Έλεγχος Εγκυρότητας Δεδομένων:

- Επαληθεύει αν το μοντέλο είναι έγκυρο μέσω ModelState.IsValid. Αν όχι, επιστρέφει τον χρήστη στη φόρμα με μήνυμα αποτυχίας.

### Δημιουργία Νέου Χρήστη:

- Δημιουργείται ένα νέο αντικείμενο **User** με δεδομένα από το αντικείμενο **Client.User**.

### Σύνδεση Χρήστη με Πελάτη:

- Το **UserId** του νέου χρήστη αντιστοιχίζεται στον πελάτη.

### Αποθήκευση στη Βάση:

- Ο πελάτης αποθηκεύεται στη βάση μέσω Clients.Add(client) και SaveChangesAsync().

### Ενημέρωση Χρήστη:

- Αν η εγγραφή είναι επιτυχής, εμφανίζεται μήνυμα επιβεβαίωσης μέσω **TempData**. Αν αποτύχει, εμφανίζεται μήνυμα αποτυχίας.
- Η μέθοδος χρησιμοποιεί το **Entity Framework** για εύκολη διαχείριση των δεδομένων, διασφαλίζοντας ότι ο πελάτης και ο χρήστης συνδέονται σωστά.

### Παράδειγμα:

Με το πάτημα του κουμπιού τα στοιχεία με την βοήθεια του Controller και του Model θα αποθηκευτούν στην βάση όπως φαίνεται παραπάνω.

**Καταχώρηση Πελάτη**

|              |                                            |
|--------------|--------------------------------------------|
| First Name   | <input type="text" value="Pantelis"/>      |
| Last Name    | <input type="text" value="Papadopoulos"/>  |
| Username     | <input type="text" value="ppapadopoulos"/> |
| AFM          | <input type="text" value="12345733"/>      |
| Phone Number | <input type="text" value="6940599166"/>    |

Ο χρήστης καταχωρήθηκε με επιτυχία με ID: 31

### Πίνακας Users της Βάσης Δεδομένων:

|    |    |          |              |               |
|----|----|----------|--------------|---------------|
| 20 | 31 | Pantelis | Papadopoulos | ppapadopoulos |
|----|----|----------|--------------|---------------|

### Πίνακας Clients της Βάσης Δεδομένων:

|   |    |          |            |    |
|---|----|----------|------------|----|
| 8 | 11 | 12345733 | 6940599166 | 31 |
|---|----|----------|------------|----|

### 3. BillsController

#### Index():

Πραγματοποιείται το ερώτημα 1.3 (Πελάτης)  
Εξόφληση Λογαριασμού

Αρχικά, ελέγχεται αν ο χρήστης είναι πελάτης και συνδεδεμένος με έγκυρο UserId. Στη συνέχεια, γίνεται **ανάκτηση δεδομένων** από τον πίνακα Clients με χρήση του Include για φόρτωση της σχετικής οντότητας User. Εφόσον εντοπιστεί ο πελάτης, το PhoneNumber του χρησιμοποιείται για **φιλτράρισμα δεδομένων** στον πίνακα Bills μέσω Where. Η μέθοδος Include χρησιμοποιείται ξανά για φόρτωση της σχετικής οντότητας PhoneNumberNavigation, και τα αποτελέσματα επιστρέφονται ως λίστα με ToListAsync(). Έτσι, τα αντικείμενα της βάσης δεδομένων διαχειρίζονται με απλό και αποδοτικό τρόπο.

```
public async Task<IActionResult> Index()
{
    if (!IsClient())
    {
        TempData["AlertMessage"] = "You need to log in as Client!";
        return RedirectToAction("Login", "Home");
    }

    var userId = HttpContext.Session.GetString("UserId");

    if (string.IsNullOrEmpty(userId))
    {
        TempData["AlertMessage"] = "You need to log in!";
        return RedirectToAction("Index", "Home");
    }

    var client = await _context.Clients
        .Include(c => c.User)
        .FirstOrDefaultAsync(c => c.UserId.ToString() == userId);

    if (client == null)
    {
        return NotFound();
    }

    var phoneNumber = client.PhoneNumber;

    if (string.IsNullOrEmpty(phoneNumber))
    {
        TempData["AlertMessage"] = "No phone number found for this client!";
        return RedirectToAction("Index", "Home");
    }

    var bills = await _context.Bills
        .Where(b => b.PhoneNumber == phoneNumber)
        .Include(b => b.PhoneNumberNavigation)
        .ToListAsync();

    return View(bills);
}
```

#### **Παράδειγμα:**

Όπως περιγράφεται με την βοήθεια των Models και του Controller ανακτώνται μόνο οι συγκεκριμένες πληροφορίες που αφορούν έναν πελάτη.

#### Ιστορικό Κλήσεων

| Description                       |
|-----------------------------------|
| Call to 2101234567 for 5 minutes  |
| Call to 2117654321 for 10 minutes |

## Dept(int? userId):

Πραγματοποιείται το ερώτημα 3.2 (Πωλητής) Έκδοση λογαριασμού Πελάτη

```
[HttpPost]
public IActionResult Dept(int? userId)
{
    if (!userId.HasValue)
    {
        ViewData["ErrorMessage"] = "Παρακαλώ εισάγετε έναν έγκυρο κωδικό χρήστη.";
        return View("ShowClientDept");
    }

    var user = _context.Users.FirstOrDefault(u => u.UserId == userId.Value);

    if (user != null)
    {
        var bill = (from b in _context.Bills
                    join c in _context.Clients on b.PhoneNumber equals c.PhoneNumber
                    where c.UserId == user.UserId
                    select b).FirstOrDefault();

        if (bill != null)
        {
            return RedirectToAction("Details", new { id = bill.BillId });
        }

        ViewData["ErrorMessage"] = $"Δεν βρέθηκε λογαριασμός για τον χρήστη με κωδικό: {userId}";
        return View("ShowClientDept");
    }

    ViewData["ErrorMessage"] = $"Δεν βρέθηκε χρήστης με κωδικό: {userId}";
    return View("ShowClientDept");
}
```

### Λήψη του userId:

- Η μέθοδος Dept είναι ένας **HTTP POST handler** που δέχεται έναν προαιρετικό userId. Αν δεν υπάρχει τιμή για το userId (δηλαδή είναι null), εμφανίζεται μήνυμα σφάλματος στην View χρησιμοποιώντας το ViewData["ErrorMessage"].
- Η αναφορά στο ViewData επιτρέπει την αποστολή μηνυμάτων ή δεδομένων από τον **Controller** προς την **View** για να εμφανιστούν στον χρήστη.

### Αναζήτηση Χρήστη στη Βάση Δεδομένων:

- Εάν το userId υπάρχει, το σύστημα αναζητά τον χρήστη στη βάση δεδομένων μέσω του **DbContext**. Η γραμμή `_context.Users.FirstOrDefault(u => u.UserId == userId.Value)` εκτελεί ένα **query** για να βρει τον χρήστη με το συγκεκριμένο userId. Το αποτέλεσμα αποθηκεύεται στη μεταβλητή user.

## Εύρεση Λογαριασμού για τον Χρήστη:

- Αν ο χρήστης βρέθηκε, ο κώδικας συνεχίζει με μια πιο σύνθετη αναζήτηση: γίνεται **join** του πίνακα Bills με τον πίνακα Clients μέσω του πεδίου PhoneNumber και φιλτράρει τα αποτελέσματα για τον συγκεκριμένο χρήστη (c.UserId == user.UserId).
- Αυτό το **join** επιτρέπει την ανάκτηση του λογαριασμού (bill) που σχετίζεται με τον χρήστη μέσω της τηλεφωνικής του γραμμής (ή άλλου κοινότυπου πεδίου).

## Αν βρεθεί λογαριασμός (bill):

- Αν βρεθεί λογαριασμός (bill), ο χρήστης ανακατευθύνεται στη σελίδα **Details** για να δει τα στοιχεία του λογαριασμού, χρησιμοποιώντας την RedirectToAction και περνώντας το bill.BillId ως παράμετρο.

## Αν δεν βρεθεί λογαριασμός ή χρήστης:

- Εάν δεν βρεθεί λογαριασμός για τον χρήστη ή αν δεν βρεθεί ο χρήστης με το συγκεκριμένο userId, η μέθοδος προσθέτει το κατάλληλο μήνυμα σφάλματος στο ViewData["ErrorMessage"] και επιστρέφει ξανά την **View** "ShowClientDept", προκειμένου ο χρήστης να δει το μήνυμα σφάλματος.

## Παράδειγμα:

Ανακτάται μέσω της αναζήτησης ο λογαριασμός του Πελάτη με id=2

MyApp

Πελάτες ▾

Διαχειριστές ▾

Πωλητές ▾

Προβολή Λογαριασμού Πελάτη:

Κωδικός Χρήστη

Αναζήτηση

MyApp

Πελάτες ▾

Διαχειριστές ▾

Πωλητές ▾

Details

Bill

|              |            |
|--------------|------------|
| Costs        | 15.00      |
| Phone Number | 6971234567 |

## 4. ProgrammsController

### Create():

Πραγματοποιείται το ερώτημα 2.2 (Διαχειριστής) Δημιουργία Νέου Προγράμματος

Το αντικείμενο **programm** αποστέλλεται από την **View** στην **Controller** μέσω του **Model Binding** με το [Bind] attribute. Αυτό σημαίνει ότι τα πεδία της φόρμας, όπως το **ProgrammName**, **Benfits**, και **Charge**, αντιστοιχούν στις ιδιότητες του αντικειμένου **Programm**.

Εάν η κατάσταση του μοντέλου είναι έγκυρη (δηλαδή τα δεδομένα πληρούν όλους τους περιορισμούς που έχουν τεθεί), το πρόγραμμα προστίθεται στη βάση δεδομένων μέσω του **DbContext** με τη μέθοδο **\_context.Add(programm)**.

Στη συνέχεια, οι αλλαγές αποθηκεύονται με την **await \_context.SaveChangesAsync()**, η οποία είναι μια ασύγχρονη μέθοδος που αποθηκεύει το νέο πρόγραμμα στη βάση.

```
// POST: Programms/Create
// To protect from overposting attacks, enable the specific properties you want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create([Bind("ProgrammName,Benfits,Charge")] Programm programm)
{
    if (ModelState.IsValid)
    {
        _context.Add(programm);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(programm);
}

public async Task<IActionResult> Edit(string id)
{
    if (id == null)
    {
        return NotFound();
    }

    var programm = await _context.Programms
        .FirstOrDefaultAsync(m => m.ProgrammName == id);
    if (programm == null)
    {
        return NotFound();
    }

    return View(programm);
}
```

Αν όλα πάνε καλά, ο χρήστης ανακατευθύνεται στη σελίδα **Index** για να δει τη λίστα των προγραμμάτων.

Εάν το μοντέλο δεν είναι έγκυρο, επιστρέφει την ίδια φόρμα (**View**) και αποδεικνύει τα σφάλματα στην επικύρωση.

### **Παράδειγμα:**

Εισαγωγή προγράμματος:

### **Δημιουργία Προγράμματος**

Program Name

Benfits

Charge

Create

Αφού εισαχθεί το νέο πρόγραμμα επιτυχώς, εμφανίζεται η λίστα με όλα τα διαθέσιμα προγράμματα:

## Αλλαγή Χαρακτηριστικών Προγράμματος

| ProgrammName | Benfits                      | Charge |                      |
|--------------|------------------------------|--------|----------------------|
| Basic        | 100 minutes, 2GB Data        | 10.00  | <a href="#">Edit</a> |
| Gift         | 10GB                         | 0.00   | <a href="#">Edit</a> |
| Premium      | Unlimited minutes, 10GB Data | 40.00  | <a href="#">Edit</a> |
| Standard     | 300 minutes, 5GB Data        | 20.00  | <a href="#">Edit</a> |
| Unlimited    | Unlimited data and sms       | 20.00  | <a href="#">Edit</a> |

### Pergram(int? userId)

Πραγματοποιείται το ερώτημα 3.3 (Πωλητές) Αλλαγή Προγράμματος Πελάτη

```
[HttpPost]
public IActionResult Pergram(int? userId)
{
    if (!userId.HasValue)
    {
        ViewData["ErrorMessage"] = "Παρακαλώ εισάγετε έναν έγκυρο κωδικό χρήστη.";
        return View("EditClientsProgram");
    }

    var user = _context.Users.FirstOrDefault(u => u.UserId == userId.Value);

    if (user == null)
    {
        ViewData["ErrorMessage"] = $"Δεν βρέθηκε χρήστης με κωδικό: {userId}";
        return View("EditClientsProgram");
    }

    var program = (from client in _context.Clients
        join phone in _context.Phones on client.PhoneNumber equals phone.PhoneNumber
        join programm in _context.Programms on phone.ProgrammName equals programm.ProgrammName
        where client.UserId == userId.Value
        select programm).FirstOrDefault();

    if (program != null)
    {
        return RedirectToAction("Edit", new { id = program.ProgrammName });
    }

    ViewData["ErrorMessage"] = $"Δεν βρέθηκε πρόγραμμα για τον χρήστη με κωδικό: {userId}";
    return View("EditClientsProgram");
}
```

#### Έλεγχος Εισόδου:

Αρχικά, γίνεται έλεγχος αν παρέχεται έγκυρο userId. Αν το userId είναι null, επιστρέφεται μήνυμα σφάλματος (ViewData["ErrorMessage"]) και ο χρήστης ανακατευθύνεται πίσω στη φόρμα με το View("EditClientsProgram").

#### Αναζήτηση Χρήστη:

Χρησιμοποιείται το `FirstOrDefault` για την εύρεση ενός χρήστη στη βάση δεδομένων από τον πίνακα `Users`, μέσω του `_context.Users`. Αν ο χρήστης δεν βρεθεί, επιστρέφεται μήνυμα σφάλματος.

### **Αναζήτηση Προγράμματος:**

Πραγματοποιείται ένα query που συνδυάζει δεδομένα από τρεις διαφορετικούς πίνακες (`Clients`, `Phones`, `Programms`) μέσω συσχετίσεων: Οι πελάτες (`Clients`) συνδέονται με τους αριθμούς τηλεφώνου (`Phones`) μέσω του `PhoneNumber`. Οι αριθμοί τηλεφώνου συνδέονται με τα προγράμματα (`Programms`) μέσω του `ProgrammName`. Το αποτέλεσμα περιορίζεται από το `UserId` του πελάτη. Αν βρεθεί πρόγραμμα, ανακατευθύνει στη σελίδα επεξεργασίας με το `RedirectToAction("Edit", new { id = program.ProgrammName })`.

### **Μη Εύρεση Δεδομένων:**

Αν δεν βρεθεί πρόγραμμα για τον συγκεκριμένο χρήστη, επιστρέφεται μήνυμα σφάλματος και η μέθοδος επαναφέρει τον χρήστη στη φόρμα.

### **Χρήση Αντικειμένων Βάσης Δεδομένων:**

#### **DbContext και DbSet:**

Το `_context` είναι το αντικείμενο του Entity Framework που διαχειρίζεται την πρόσβαση στους πίνακες της βάσης (`Users`, `Clients`, `Phones`, `Programms`). Συνδέονται δεδομένα από πολλούς πίνακες με `join` για να επιστρέψει το σχετικό πρόγραμμα του χρήστη. Η χρήση του `where` εξασφαλίζει ότι το αποτέλεσμα φιλτράρεται με βάση το `UserId`.

### **Ανακατεύθυνση και Επιστροφή Δεδομένων:**

Αν βρεθεί το πρόγραμμα, χρησιμοποιείται το `RedirectToAction` για ανακατεύθυνση στη μέθοδο `Edit`, περνώντας το `ProgrammName` ως παράμετρο.

Εναλλακτικά, αν δεν υπάρχουν δεδομένα, επιστρέφεται η ίδια φόρμα με μήνυμα σφάλματος, χρησιμοποιώντας το `ViewData` για την επικοινωνία του μηνύματος στον χρήστη.

## Παράδειγμα:

Μετά την εύρεση του προγράμματος και την επεξεργασία του αλλάζουν και τα δεδομένα της συγκεκριμένης εγγραφής στην βάση δεδομένων.

## Edit

Program Name

Standard

Benefits

300 minutes, 5GB Data

Charge

20.00

Save

[Back to List](#)

**Edit(string id, [Bind("ProgrammName,Benefits,Charge")] Programm programm):**

Πραγματοποιείται το ερώτημα 2.3 (Διαχειριστής) Αλλαγή Χαρακτηριστικών Προγράμματος

```
// POST: Programms/Edit/5
// To protect from overposting attacks, enable the specific properties you want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(string id, [Bind("ProgrammName,Benefits,Charge")] Programm programm)
{
    if (id != programm.ProgrammName)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            _context.Update(programm);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!ProgrammExists(programm.ProgrammName))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }

        return RedirectToAction(nameof(Index));
    }
    return View(programm);
}
```

1. **Έλεγχος Ταύτισης ID:** Ελέγχεται αν το id που παρέχεται στη διαδρομή URL ταιριάζει με το ProgrammName του αντικειμένου programm. Αν δεν ταιριάζουν, επιστρέφεται σφάλμα NotFound().

2. **Επικύρωση Δεδομένων:** Η μέθοδος `ModelState.IsValid` διασφαλίζει ότι τα δεδομένα που εισάγονται πληρούν τους κανόνες επικύρωσης που έχουν οριστεί στο μοντέλο `Programm`.
3. **Ενημέρωση Δεδομένων:** Αν τα δεδομένα είναι έγκυρα, το αντικείμενο `programm` ενημερώνεται στη βάση δεδομένων με τη χρήση της μεθόδου `_context.Update(programm)`. Η κλήση `await _context.SaveChangesAsync()` αποθηκεύει τις αλλαγές.
4. **Χειρισμός Συγκρούσεων Ενημέρωσης:** Αν προκύψει εξαίρεση `DbUpdateConcurrencyException` (π.χ. όταν υπάρχει πρόβλημα ταυτόχρονης πρόσβασης στη βάση), ελέγχεται αν το αντικείμενο με το συγκεκριμένο `ProgrammName` υπάρχει ακόμα στη βάση. Αν δεν υπάρχει, επιστρέφεται σφάλμα `NotFound()`. Αν υπάρχει, η εξαίρεση προωθείται περαιτέρω.
5. **Ανακατεύθυνση ή Επιστροφή στη Φόρμα:** Αν η επεξεργασία ολοκληρωθεί επιτυχώς, ο χρήστης ανακατευθύνεται στη σελίδα `Index`. Αν τα δεδομένα δεν είναι έγκυρα, επιστρέφεται η ίδια φόρμα μαζί με τα σφάλματα επικύρωσης.

### Χρήση Αντικειμένων Βάσης Δεδομένων:

1. **DbContext και DbSet:** Το `_context` χρησιμοποιείται για την πρόσβαση στον πίνακα `Programms` της βάσης μέσω του `DbSet`.
2. **Ενημέρωση Οντοτήτων:** Το αντικείμενο `programm` ενημερώνεται στη βάση με τη μέθοδο `_context.Update(programm)`, που δηλώνει στο Entity Framework ότι το αντικείμενο έχει αλλάξει.
3. **Αποθήκευση Αλλαγών:** Η μέθοδος `SaveChangesAsync()` αποθηκεύει τις αλλαγές ασύγχρονα, εξασφαλίζοντας καλύτερη απόδοση.

### Παράδειγμα:

Όπως αναφέρθηκε και παραπάνω πατώντας το `save` ολοκληρώνεται με την βοήθεια των `Models` και του `Controller` η τροποποίηση του προγράμματος.

## Edit

Program Name

Benefits

Charge

[Back to List](#)

## 5. SellersController

Πραγματοποιείται το ερώτημα 2.1 (Διαχειριστής) Δημιουργία νέου Πωλητή

Create():

```
// POST: Sellers/Create
// To protect from overposting attacks, enable the specific properties you want to bind to.
// For more details, see http://go.microsoft.com/fwlink/?LinkId=317598.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(Seller seller)
{
    if (ModelState.IsValid)
    {
        // Δημιουργία νέου χρήστη
        var user = new User
        {
            FirstName = seller.User.FirstName,
            LastName = seller.User.LastName,
            Username = seller.User.Username
        };

        // Δημιουργία νέου πωλητή
        seller.UserId = user.UserId;
        _context.Sellers.Add(seller);
        await _context.SaveChangesAsync();

        TempData["SuccessMessage"] = $"Ο χρήστης καταχωρήθηκε με επιτυχία με ID: {seller.UserId}";
    }
    else
    {
        TempData["SuccessMessage"] = "Ο χρήστης δεν καταχωρήθηκε!";
    }

    return View(seller);
}
```

Αρχικά, γίνεται έλεγχος αν το μοντέλο είναι έγκυρο με τη χρήση του ModelState.IsValid. Αν το μοντέλο είναι έγκυρο, δημιουργείται πρώτα ένα αντικείμενο User για τον πωλητή. Οι ιδιότητες

FirstName, LastName και Username του User αντιστοιχούν στα δεδομένα που δόθηκαν από τη φόρμα μέσω του seller.User.

Μετά τη δημιουργία του αντικειμένου User, το UserId του πωλητή (seller.UserId) ανατίθεται στο UserId του χρήστη, ώστε να υπάρχει σύνδεση μεταξύ των δύο οντοτήτων. Στη συνέχεια, το αντικείμενο Seller προστίθεται στη βάση δεδομένων με τη χρήση του `_context.Sellers.Add(seller)` και οι αλλαγές αποθηκεύονται με την ασύγχρονη μέθοδο `await _context.SaveChangesAsync()`.

Εφόσον η εγγραφή ολοκληρωθεί επιτυχώς, αποθηκεύεται ένα μήνυμα επιτυχίας στο TempData, ενημερώνοντας τον χρήστη ότι ο πωλητής δημιουργήθηκε με επιτυχία. Αν το μοντέλο δεν είναι έγκυρο, αποθηκεύεται αντίστοιχα ένα μήνυμα αποτυχίας στο TempData και η φόρμα επιστρέφεται στον χρήστη.

### Χρήση Αντικειμένων Βάσης Δεδομένων:

1. DbContext και DbSet: Η βάση δεδομένων προσεγγίζεται μέσω του αντικειμένου `_context`, το οποίο αντιπροσωπεύει το DbContext της εφαρμογής. Το Sellers είναι ένα DbSet που αντιστοιχεί στον πίνακα των πωλητών στη βάση δεδομένων.
2. Προσθήκη Δεδομένων: Το νέο αντικείμενο Seller προστίθεται στη βάση μέσω της μεθόδου `_context.Sellers.Add(seller)`. Η μέθοδος `SaveChangesAsync()` αποθηκεύει τις αλλαγές.
3. Συσχέτιση Οντοτήτων: Το UserId του πωλητή ανατίθεται από το αντίστοιχο UserId του χρήστη, διασφαλίζοντας τη σύνδεση μεταξύ των δύο πινάκων.
4. Ασφάλεια: Η χρήση του ModelState.IsValid εξασφαλίζει ότι τα δεδομένα που εισάγονται πληρούν τους κανόνες επικύρωσης του μοντέλου, αποτρέποντας σφάλματα και μη έγκυρα δεδομένα στη βάση.

### Παράδειγμα:

#### Δημιουργία Πωλητή

First Name

Vasiliki

Last Name

Vasilikou

Username

vasvasil

Δημιουργία

Ο χρήστης καταχωρήθηκε με επιτυχία με ID: 30

### Πίνακας Users:

|    |    |          |           |          |
|----|----|----------|-----------|----------|
| 19 | 30 | Vasiliki | Vasilikou | vasvasil |
|----|----|----------|-----------|----------|

## 6. CallsController

Πραγματοποιείται το ερώτημα 1.2 (Πελάτης) Προβολή ιστορικού κλήσεων

### Index():

Αρχικά, η μέθοδος Index ελέγχει αν ο χρήστης είναι συνδεδεμένος ως "Client". Αν ο χρήστης δεν είναι συνδεδεμένος, χρησιμοποιείται το TempData για να εμφανιστεί ένα μήνυμα σφάλματος και γίνεται ανακατεύθυνση στην σελίδα σύνδεσης. Αυτός ο έλεγχος γίνεται με τη μέθοδο IsClient(), η οποία πιθανώς αναγνωρίζει τον ρόλο του χρήστη είτε από την Session είτε από τη βάση δεδομένων.

Στη συνέχεια, το userId ανακτάται από την Session του χρήστη με τη μέθοδο HttpContext.Session.GetString("UserId"). Εάν το userId είναι null ή κενό, σημαίνει ότι ο χρήστης δεν είναι συνδεδεμένος και τον ανακατευθύνει πίσω στην αρχική σελίδα με ένα μήνυμα σφάλματος. Αυτή η διαδικασία διασφαλίζει ότι μόνο οι συνδεδεμένοι χρήστες μπορούν να προχωρήσουν.

Αφού γίνει ο έλεγχος σύνδεσης, ο κώδικας χρησιμοποιεί το Entity Framework για να ανακτήσει δεδομένα από τη βάση. Συγκεκριμένα, γίνεται αναζήτηση του πελάτη με το συγκεκριμένο userId στη βάση δεδομένων μέσω της DbContext (\_context.Clients). Χρησιμοποιείται η μέθοδος FirstOrDefaultAsync για να βρει τον πελάτη με το συγκεκριμένο UserId. Εάν ο πελάτης δεν βρεθεί, επιστρέφεται το σφάλμα NotFound(). Αν βρεθεί, το επόμενο βήμα είναι να ελεγχθεί αν υπάρχει καταχωρημένος αριθμός τηλεφώνου για τον πελάτη. Αν δεν υπάρχει, το πρόγραμμα ενημερώνει τον χρήστη με ένα μήνυμα σφάλματος.

Εφόσον υπάρχει αριθμός τηλεφώνου, το πρόγραμμα χρησιμοποιεί τον αριθμό αυτόν για να ανακτήσει τις κλήσεις του πελάτη από τον πίνακα Bills, συνδυάζοντας τα δεδομένα από το Bills

```
public async Task<IActionResult> Index()
{
    if (!IsClient())
    {
        TempData["AlertMessage"] = "You need to log in as Client!";
        return RedirectToAction("Login", "Home");
    }

    var userId = HttpContext.Session.GetString("UserId");

    if (string.IsNullOrEmpty(userId))
    {
        TempData["AlertMessage"] = "You need to log in!";
        return RedirectToAction("Index", "Home");
    }

    var client = await _context.Clients
        .Include(c => c.User)
        .FirstOrDefaultAsync(c => c.UserId.ToString() == userId);

    if (client == null)
    {
        return NotFound();
    }

    var phoneNumber = client.PhoneNumber;

    if (string.IsNullOrEmpty(phoneNumber))
    {
        TempData["AlertMessage"] = "No phone number found for this client!";
        return RedirectToAction("Index", "Home");
    }

    var calls = await _context.Bills
        .Where(b => b.PhoneNumber == phoneNumber)
        .SelectMany(b => b.Calls)
        .ToListAsync();

    return View(calls);
}
```

και το Calls μέσω της σχέσης τους στη βάση. Αυτό γίνεται με την χρήση του LINQ query και της μεθόδου Where για να φιλτράρει τα αποτελέσματα που σχετίζονται με το συγκεκριμένο phoneNumber. Οι κλήσεις που αντιστοιχούν σε αυτόν τον αριθμό τηλεφώνου ανακτώνται μέσω της μεθόδου SelectMany, η οποία επιστρέφει όλα τα σχετικά αντικείμενα από τον πίνακα Calls.

Τελικά, οι κλήσεις που ανακτήθηκαν επιστρέφονται στην View για να παρουσιαστούν στον χρήστη.

### Παράδειγμα:

#### Ιστορικό Κλήσεων

| Description                       |
|-----------------------------------|
| Call to 2101234567 for 5 minutes  |
| Call to 2117654321 for 10 minutes |